CHAPTER 3
ASP.NET Core Platform

CONTENT

- ASP.NET Core Platform
- Prepare the project
- Create custom Middleware
- Configuration Middleware

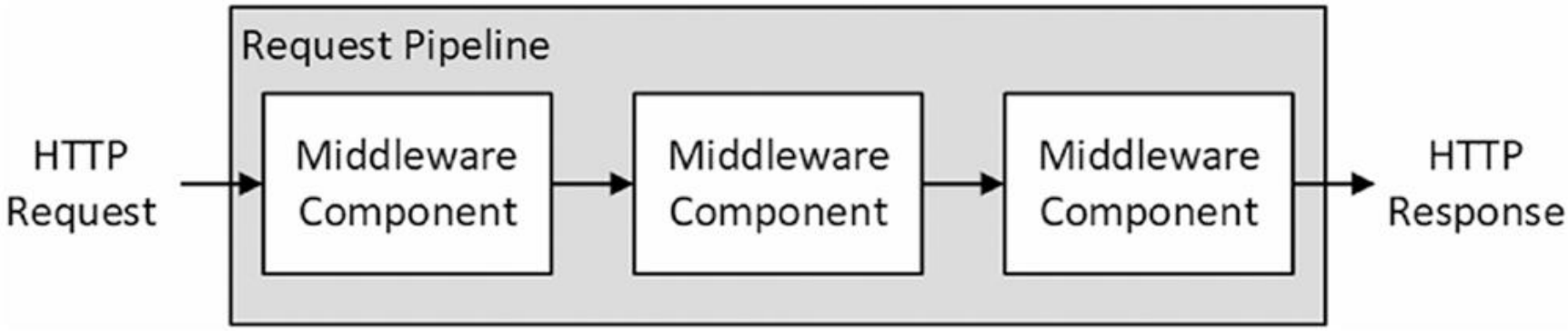
ASP.NET CORE PLATFORM

ASP.NET Core Platform

- Understanding Middleware and the Request Pipeline:
  - The purpose of the ASP.NET Core platform is to receive HTTP requests and send responses to them, which ASP.NET Core delegates to middleware components. Middleware components are arranged in a chain, known as the request pipeline.
  - When a new HTTP request arrives, the ASP.NET Core platform creates an object that describes it and a corresponding object that describes the response that will be sent in return. These objects are passed to the first middleware component in the chain, which inspects the request and modifies the response.

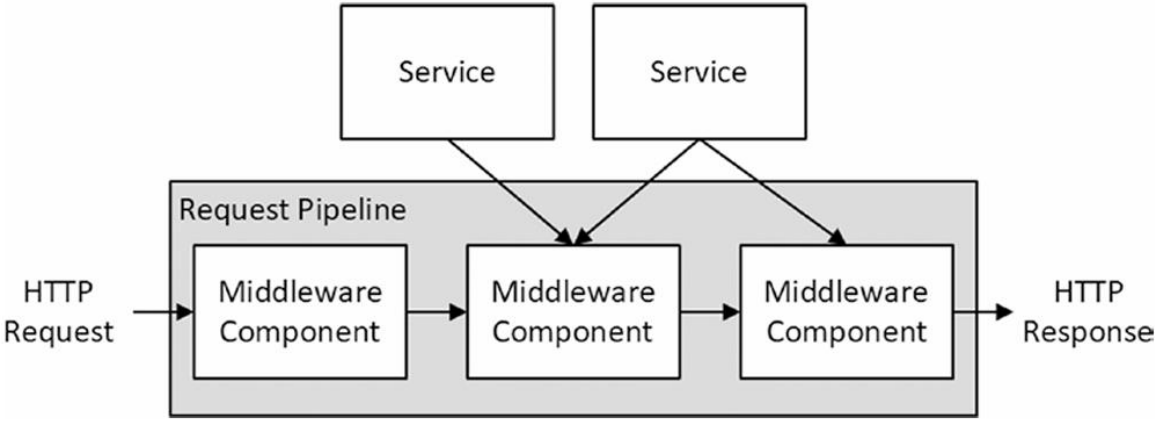
ASP.NET Core Platform

- Understanding Middleware and the Request Pipeline:
  - The request is then passed to the next middleware component in the chain, with each component inspecting the request and adding to the response. Once the request has made its way through the pipeline, the ASP.NET Core platform sends the response



ASP.NET Core Platform

- Understanding Services:
  - Services are objects that provide features in a web application. Any class can be used as a service, and there are no restrictions on the features that services provide. What makes services special is that they are managed by ASP.NET Core, and a feature called **dependency injection** makes it possible to easily access services anywhere in the application, including middleware components



PREPARE THE PROJECT

Prepare the project

- To prepare for this chapter, I am going to create a new project named Platform, using the template that provides the minimal ASP.NET Core setup. Open a new PowerShell command prompt from the Windows Start menu and run the commands
  - dotnet new globaljson --sdk-version 6.0.100 --output **Platform**
  - dotnet new web --no-https --output Platform --framework net6.0
  - dotnet new sln -o Platform
  - dotnet sln Platform add Platform

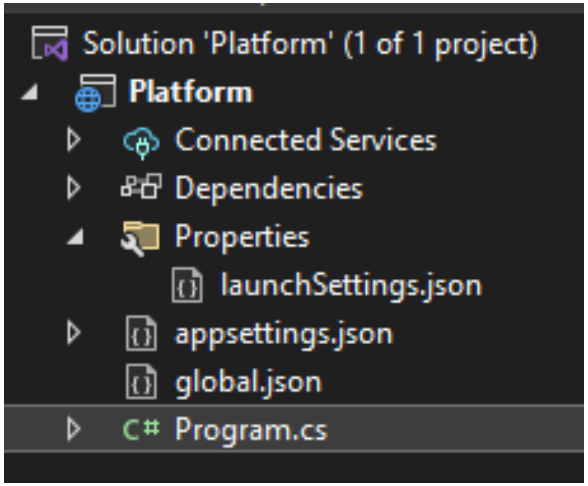
Prepare the project

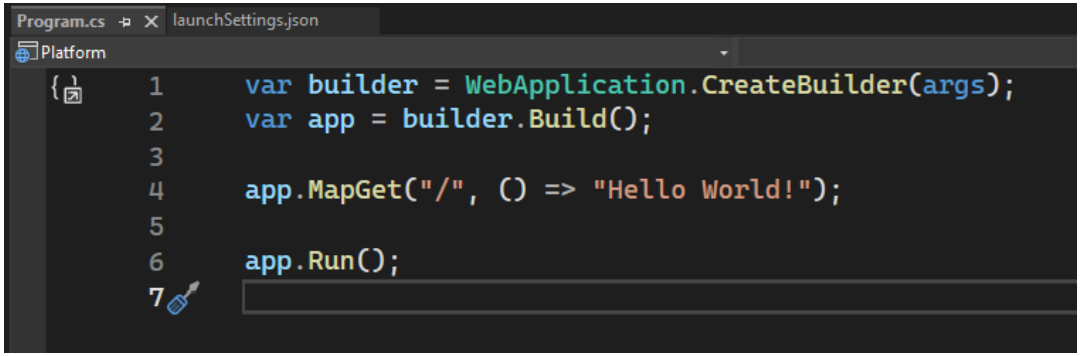
Table 12-3. *The Files and Folders in the Example Project*

Name	Description
`appsettings.json`	This file is used to configure the application, as described in Chapter 15.
`appsettings.Development.json`	This file is used to define configuration settings that are specific to development, as explained in Chapter 15.
`bin`	This folder contains the compiled application files. Visual Studio hides this folder.
`global.json`	This file is used to select a specific version of the .NET Core SDK.
`Properties/launchSettings.json`	This file is used to configure the application when it starts.
`obj`	This folder contains the intermediate output from the compiler. Visual Studio hides this folder.
`Platform.csproj`	This file describes the project to the .NET Core tools, including the package dependencies and build instructions, as described in the "Understanding the Project File" section. Visual Studio hides this file, but it can be edited by right-clicking the project item in the Solution Explorer and selecting Edit Project File from the pop-up menu.
`Platform.sln`	This file is used to organize projects. Visual Studio hides this folder.
`Program.cs`	This file is the entry point for the ASP.NET Core platform and is used to configure the platform, as described in the "Understanding the Entry Point" section

Understanding the Entry Point

- The **Program.cs** file contains the code statements that are executed when the application is started and that are used to configure the ASP.NET platform and the individual frameworks it supports. Here is the content of the Program.cs file in the example project:





Understanding the Entry Point

- This file contains only top-level statements. The first statement calls the WebApplication.CreateBuilder and assigns the result to a variable named builder:

```
...
var builder = WebApplication.CreateBuilder(args);
...
```

- This method is responsible for **setting up the basic features** of the ASP.NET Core platform, including creating services responsible for configuration data and logging. This method also sets up the **HTTP server, named Kestrel**, that is used to receive HTTP requests

Understanding the Entry Point

- The result from the CreateBuilder method is a WebApplicationBuilder object, which is used to register additional services, although none are defined at present. The WebApplicationBuilder class defines a Build method that is used to finalize the initial setup:

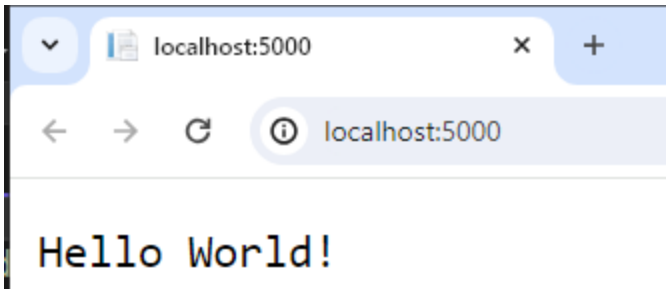
```
...
var app = builder.Build();
...
```

Understanding the Entry Point

- The result of the Build method is a WebApplication object, which is used to set up middleware components. The template has set up one middleware component, using the MapGet extension method:

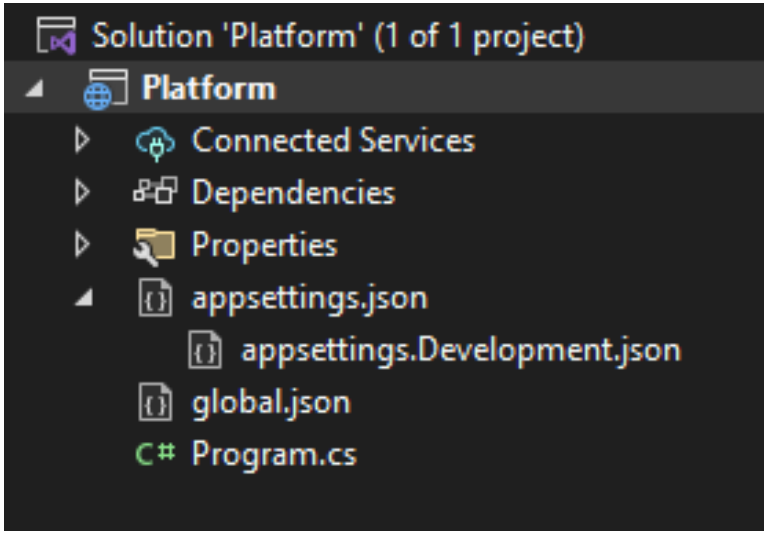
```
...
app.MapGet("/", () => "Hello World!");
...
```

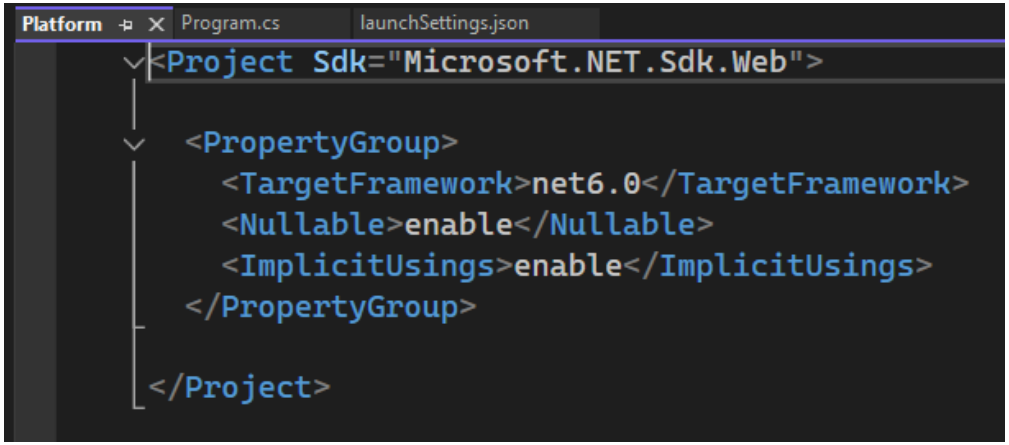
- MapGet is an extension method for the IEndpointRouteBuilder interface, which is implemented by the WebApplication class, and which sets up a function that will **handle HTTP requests with a specified URL path**. In this case, the function responds to requests for the default URL path, which is denoted by /, and the function responds to all requests by **returning a simple string response**



Understanding the Project File

- The Platform.csproj file, known as the project file, contains the information that .NET Core uses to build the project and keep track of dependencies.





CREATING CUSTOM MIDDLEWARE

Creating Custom Middleware

- As mentioned, Microsoft provides various middleware components for ASP.NET Core that handle the features most commonly required by web applications. You can also create your own middleware, which is a useful way to understand how ASP.NET Core works, even if you use only the standard components in your projects. The key method for creating middleware is **Use**

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    if (context.Request.Method == HttpMethods.Get
    && context.Request.Query["custom"] == "true")
    {
        context.Response.ContentType = "text/plain";
        await context.Response.WriteAsync("Custom Middleware \n");
    }
    await next();
});

app.MapGet("/", () => "Hello World!");

app.Run();
```

WriteAsync method to add a string to the body of the response

Creating Custom Middleware

- As mentioned, Microsoft provides various middleware components for ASP.NET Core that handle the features most commonly required by web applications. You can also create your own middleware, which is a useful way to understand how ASP.NET Core works, even if you use only the standard components in your projects. The key method for creating middleware is **Use**

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    if (context.Request.Method == HttpMethods.Get
    && context.Request.Query["custom"] == "true")
    {
        context.Response.ContentType = "text/plain";
        await context.Response.WriteAsync("Custom Middleware \n");
    }
    await next();
});

app.MapGet("/", () => "Hello World!");

app.Run();
```

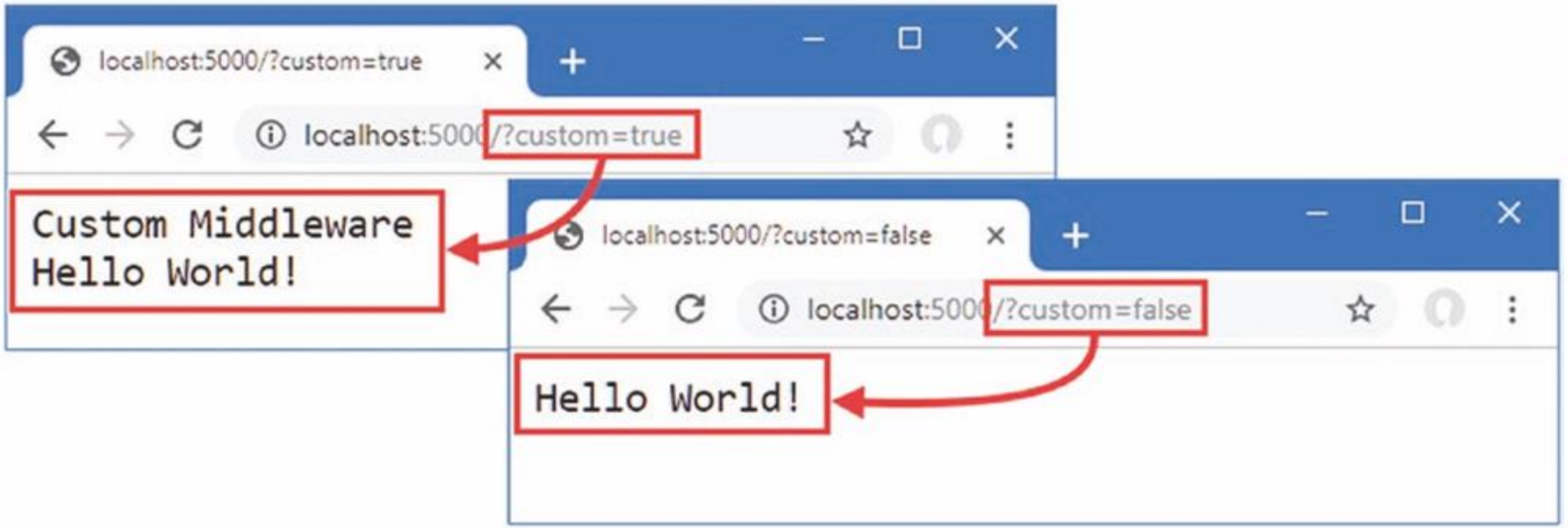
The second argument to the middleware is the function conventionally named next that tells ASP.NET Core to pass the request to the next component in the request pipeline

Creating Custom Middleware

- The **Use** method registers a middleware component that is typically expressed as a lambda function that receives each request as it passes through the pipeline (there is another method used for classes, as described in the **next section**).
- The arguments to the lambda function are **an HttpContext object and a function** that is invoked to tell ASP.NET Core to pass the request to the **next middleware** component in the pipeline.
- The HttpContext object describes the HTTP request and the HTTP response and provides additional context, including details of the user associated with the request.

Creating Custom Middleware

Start ASP.NET Core using the `dotnet run` command and use a browser to request `http://localhost:5000/?custom=true`. You will see that the new middleware function writes its message to the response before passing on the request to the next middleware component, as shown in Figure 12-5. Remove the query string, or change `true` to `false`, and the middleware component will pass on the request without adding to the response.



Defining Middleware Using a Class

- Defining middleware using lambda functions is convenient, but it can lead to a long and complex series of statements in the Program.cs file and makes it hard to reuse middleware in different projects.
- Middleware can also be defined using classes. Add a class file named **QueryStringMiddleware.cs** to the Platform folder and add the code

Defining Middleware Using a Class

```
namespace Platform
{
    public class QueryStringMiddleware
    {
        private RequestDelegate next;
        public QueryStringMiddleware(RequestDelegate nextDelegate)
        {
            next = nextDelegate;
        }
        public async Task Invoke(HttpContext context)
        {
            if (context.Request.Method == HttpMethods.Get
            && context.Request.Query["custom"] == "true")
            {
                if (!context.Response.HasStarted)
                {
                    context.Response.ContentType = "text/plain";
                }
                await context.Response.WriteAsync("Class-based Middleware \n");
            }
            await next(context);
        }
    }
}
```

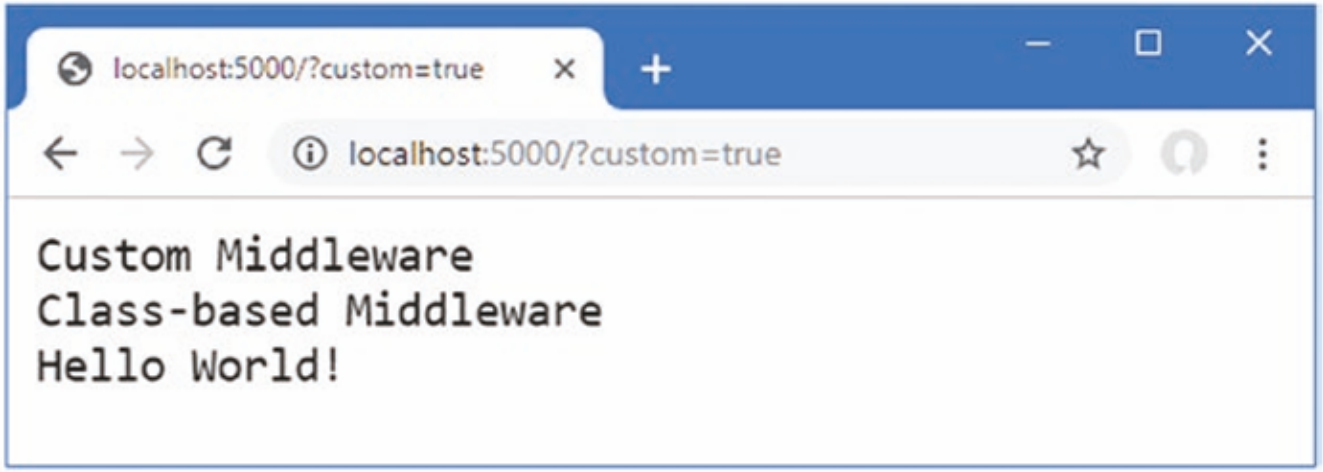
WriteAsync method to add a string to the body of the response

Defining Middleware Using a Class

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    if (context.Request.Method == HttpMethods.Get
    && context.Request.Query["custom"] == "true")
    {
        context.Response.ContentType = "text/plain";
        await context.Response.WriteAsync("Custom Middleware \n");
    }
    await next();
});

app.UseMiddleware<Platform.QueryStringMiddleware>();

app.MapGet("/", () => "Hello World!");
app.Run();
```

Defining Middleware Using a Class

Use the `dotnet run` command to start ASP.NET Core and use a browser to request `http://localhost:5000/?custom=true`. You will see the output from both middleware components, as shown in Figure 12-6.



UNDERSTANDING THE RETURN PIPELINE PATH

Return Pipeline Path

- Middleware components can modify the HTTPResponse object after the next function has been called, as shown by the new middleware below

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    await next();
    await context.Response
    .WriteAsync($"\nStatus Code: {context.Response.StatusCode}");
});

…
```

Return Pipeline Path

- The new middleware immediately calls the next method to pass the request along the pipeline and then uses the WriteAsync method to add a string to the response body. This may seem like an odd approach, but it allows middleware to make changes to the response **before and after** it is passed along the request pipeline by defining statements before and after the next function is invoked, as illustrated by Figure 12-7.

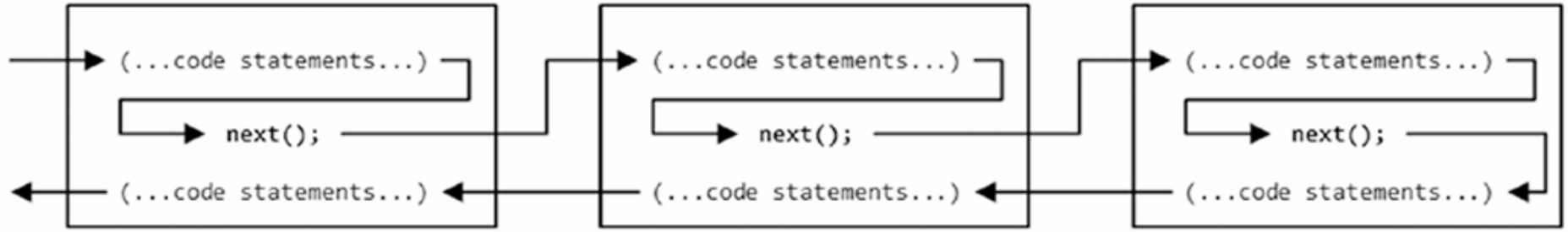
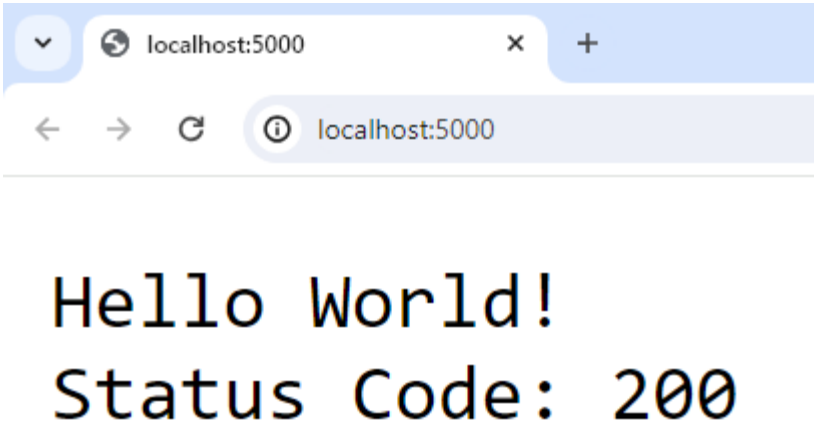


Figure 12-7. *Passing request and responses through the ASP.NET Core pipeline*

Return Pipeline Path

- Middleware can operate before the request is passed on, after the request has been processed by other components, or both. The result is that several middleware components collectively contribute to the response that is produced, each providing some aspect of the response or providing some feature or data that is used later in the pipeline.



SHORT-CIRCUITING THE REQUEST PIPELINE

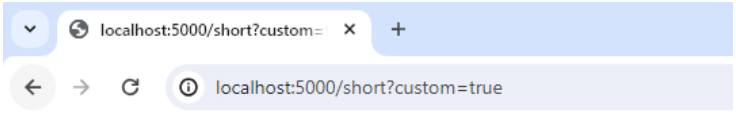
Short-Circuiting the Request Pipeline

- Components that generate complete responses can choose **not to call the next function** so that **the request isn't passed on**. Components that don't pass on requests are said to **short-circuit the pipeline**, which is what the new middleware component shown below does for requests that target the /short URL

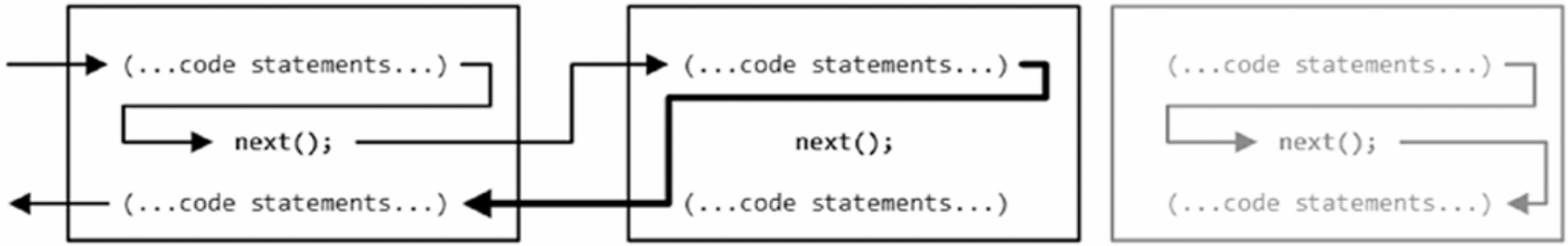
```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    await next();
    await context.Response
    .WriteAsync($"\nStatus Code: {context.Response.StatusCode}");
});

app.Use(async (context, next) => {
    if (context.Request.Path == "/short")
    {
        await context.Response
        .WriteAsync($"Request Short Circuited");
    }
    else
    {
        await next();
    }
});
```

Short-Circuiting the Request Pipeline

- The new middleware checks the Path property of the HttpRequest object to see whether the request is for the /short URL; if it is, it calls the WriteAsync method without calling the next function. To see the effect, restart ASP.NET Core and use a browser to request http://localhost:5000/short?custom=true, which will produce the output shown below:




```
Request Short Circuited
Status Code: 200
```



CREATING PIPELINE BRANCHES

Creating Pipeline Branches

- The **Map method** is used to create a section of pipeline that is used to process requests for specific URLs, creating a separate sequence of middleware components

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

((IApplicationBuilder)app).Map("/branch", branch => {
    branch.UseMiddleware<Platform.QueryStringMiddleware>();
    branch.Use(async (HttpContext context, Func<Task> next) => {
        await context.Response.WriteAsync($"Branch Middleware");
    });
});

app.UseMiddleware<Platform.QueryStringMiddleware>();
app.MapGet("/", () => "Hello World!");
app.Run();
```

Creating Pipeline Branches

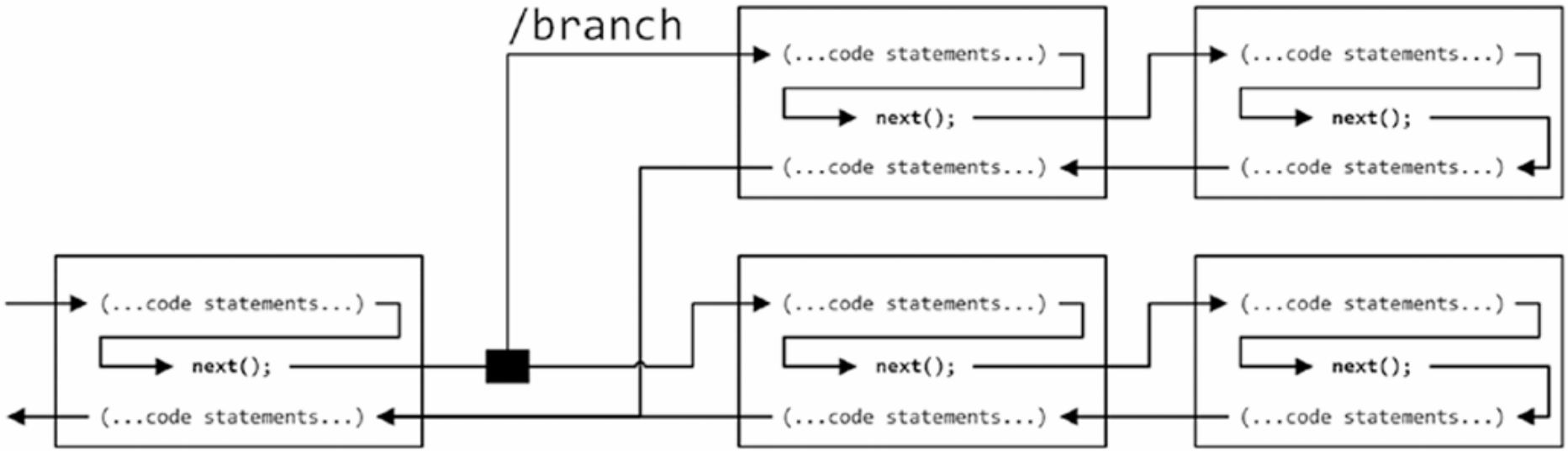
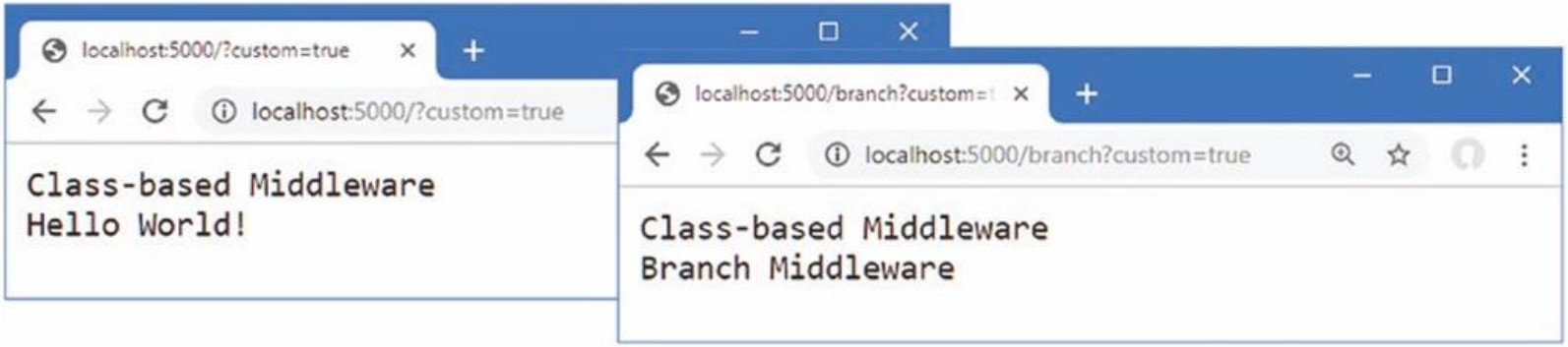


Figure 12-11. *Adding a branch to the request pipeline*



CREATING TERMINAL MIDDLEWARE

Creating Terminal Middleware

- Terminal middleware never forwards requests to other components and always marks the end of the request pipeline. There is a terminal middleware component in the Program.cs file, as shown here

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

((IApplicationBuilder)app).Map("/branch", branch => {
    branch.UseMiddleware<Platform.QueryStringMiddleware>();
    branch.Use(async (HttpContext context, Func<Task> next) => {
        await context.Response.WriteAsync($"Branch Middleware");
    });
});

app.UseMiddleware<Platform.QueryStringMiddleware>();
app.MapGet("/", () => "Hello World!");
app.Run();
```

Creating Terminal Middleware

- ASP.NET Core supports the Run method as a convenience feature for creating terminal middleware, which makes it obvious that a middleware component won't forward requests and that a deliberate decision has been made not to call the next function

```
var builder = WebApplication.CreateBuilder(args);
 var app = builder.Build();
 ((IApplicationBuilder)app).Map("/branch", branch => {
 branch.UseMiddleware<Platform.QueryStringMiddleWare>();

 branch.Run(async (context) => {
 await context.Response.WriteAsync($"Branch Middleware");
 });
 });

 app.UseMiddleware<Platform.QueryStringMiddleWare>();
 app.MapGet("/", () => "Hello World!");
 app.Run();
```